

그리드 리뉴얼 — 자료구조 · encounter · 트리거 설계

날짜: 2026-05-02 선행: 프로젝트 전반 메커니즘.pdf , 이벤트 시스템 구현 - 안 4 (kind 분리 + chain).pdf

목적: 현 노드 그래프 맵을 좌표 기반 그리드로 교체하면서, 타일이 encounter 의 단일 출처가 되는 모델을 정리. 카드 전투 / EventManager / 회귀 / 세이브 등 기존 시물은 그대로.

§0. 결정 사항 요약 (확정)

- 공간 시스템만 교체 — BattleManager / EventManager / 회귀 / 세이브 그대로.
- 좌표 기반 그리드 (Vector2i 키), 그래프 구조 안 사용. 모든 게 그리드라 그래프의 일반성 비용 부담.
- 타일 = encounter 컨테이너 — 한 타일에 여러 encounter 가능, 각자 트리거 조건.
- 트리거 타입 풀: on_enter / on_investigate / on_condition (+ 글로벌은 별).
- EVENTS 풀 = 라이브러리 — event_id 참조 대상. 위치별 트리거는 타일이 들고 있음.
- 카드 전투 그대로 재활용 — SRPG 같은 거 안 함. BattleManager 그대로.
- EventManager 에 combat kind 추가 — chain 안에서 전투 호출 가능.
- 매 회차 visited / investigated / once_per:internal_run 리셋. once_per: big_run 은 영속.
- (deferred) 주인공의 회차 인식 = 같은 encounter 두 번째 만남 시 다른 라인. 후속.

§1. 배경 — 그래프 → 그리드 동기

현 TEST_MAP_GRAPH (6 노드 hex 사이클) 의 한계: - 공간감 부재 — 6개 노드 위 점프식 이동, 풍경의 무게가 안 실림. - 탐험 결의 빈약 — 인접 검사만, 탐험·발견의 게임플레이 단위가 없음. - 시간 압박 부재 — 노드는 그냥 한 점, 시간 / 자원 결의 표현이 어색. - narrative 와 결합 약함 — 메모리상 "회차 / 영원회귀 / 알 / 융합" 의 공간적 펼쳐짐 자리 부족.

그리드로 가면: - 4방향 이동 = 공간감 직관. - step / day / 식량 같은 자원 압박 자연. - 타일 위 encounter 분산 = 매 발걸음 = 결정. - 메모리의 narrative 가 타일 위에 펼쳐짐 (1구역 / 2구역 / 거점 / 쉼터).

§2. 좌표 vs 그래프 — 결정 근거

연산	좌표 기반	그래프 기반
이동 (한 칸)	pos += dir, 0(1)	connections 0(1)
범위 (거리 N 이내)	$ dx + dy \leq N$	BFS depth N
AOE 패턴 (3×3)	offset + 좌표 덧셈	BFS + 패턴 매칭

시야 (line-of-sight)
저장 (16×12)

Bresenham
192 항목

BFS + 특수 처리
192 노드 + ~600 엣지

결정: 좌표 기반. - 모든 게 그리드라 그래프의 일반성 (비-격자 토폴로지) 안 씬 - 그리드 연산이 산술로 정리됨, 코드 명료 - 벽 / 통과 불가능 terrain 필드로 강제 (자료구조 원칙 통과) - 단방향 / 포탈 같은 특수 케이스는 별 dict (드물게 사용)

→ 다른 클로드의 "그래프 > 그리드" 통찰은 정확하나 우리 게임에선 일반성이 비용.

§3. 핵심 모델 — 타일 = encounter 컨테이너

```
tile = {
  # 정적
  pos: Vector2i,
  terrain: String,          # "grass" | "path" | "marsh" | "wall" | "water" |

  # 회차 안 동적 (매 회차 리셋)
  visited: bool,
  investigated: bool,

  # encounter 풀 (0~N 개)
  encounters: [
    {
      id: String,           # seen 카운트 키
      kind: "event"|"combat"|"research"|"campsite"|"secret",
      event_id: String?,    # kind="event" 시 EVENTS 풀 참조
      enemy_id: String?,    # kind="combat" 시 ENEMIES 풀 참조
      unlock_tool: String?, # kind="secret" 시 도구 필요

      trigger: {
        type: "on_enter"|"on_investigate"|"on_condition",
        flags_required: [String]?,
        flags_forbidden: [String]?,
        # type별 추가 필드
      },

      once_per: "big_run"|"internal_run"|null,
    },
    ...
  ]
}
```

탐험 = 타일 진입 (또는 조사 액션) = encounter 평가 + 트리거.

§4. 자료구조 — As-Is / To-Be

4-1. 폐기

```
# game_data.gd
const TEST_MAP_GRAPH = { 1: {...connections...}, ... }

# RunManager.run_data
"map": Dictionary          # 노드 사본
"current_node_id": int     # 노드 ID
```

4-2. 신설

```
# game_data.gd (또는 world_data.gd 분리)
const WORLD_TERRAIN = [
    "FFFFCCCCCCCCFFFF",    # 0
    "FGGGGGGGGGGGGGGGF",
    ...
] # 16×12 또는 N×M

const TERRAIN_RULES = {
    "G": {"name": "풀밭", "step_cost": 1.0, "passable": true},
    "P": {"name": "길", "step_cost": 0.5, "passable": true},
    "M": {"name": "늪", "step_cost": 2.0, "passable": true},
    "F": {"name": "숲", "passable": false},
    "C": {"name": "절벽", "passable": false, "unlock_tool": "climb"},
    "W": {"name": "물", "passable": false, "unlock_tool": "swim"},
}

# 타일 위 encounter 정의
const TILE_ENCOUNTERS = {
    Vector2i(3, 4): [
        {
            id: "intro_at_3_4",
            kind: "event",
            event_id: "intro_speech",
            trigger: {type: "on_enter"},
            once_per: "internal_run"
        }
    ],
    Vector2i(7, 5): [
        {
            id: "first_combat",
            kind: "combat",
            enemy_id: "larva",
            trigger: {type: "on_enter"},
            once_per: "internal_run"
        }
    ]
}
```

```

    },
    {
        id: "hidden_letter_at_7_5",
        kind: "event",
        event_id: "find_letter",
        trigger: {type: "on_investigate"},
        once_per: "big_run"
    },
    ],
    ...
}

# RunManager.run_data
"player_pos": Vector2i,
"day": int,
"steps_remaining": float,
"steps_per_day": int,
"visited_tiles": Dictionary,          # Vector2i → bool
"investigated_tiles": Dictionary,     # Vector2i → bool
"seen_this_run": Dictionary,          # encounter_id → bool (once_per: internal_run)

```

4-3. EVENTS 풀의 새 결 — 라이브러리

기존 EVENTS 의 두 역할 중 하나가 떨어져나감: - ~~위치 트리거 매치~~ → 타일이 들고 있음 - 라이브러리
(event_id 참조 대상) ← 그대로 유지

```

const EVENTS = {
    # 글로벌 트리거 - run_start 등 위치 무관
    "intro_speech": {
        "kind": "dialogue",
        "trigger": {"type": "run_start"},
        "lines": [...],
    },

    # 타일 encounter 가 event_id 로 참조 (trigger 필드 X)
    "find_letter": {
        "kind": "dialogue",
        "lines": [...],
    },
    "first_intro_chain_a": {
        "kind": "dialogue",
        "lines": [...],
        "next": "first_intro_chain_b",
    },
    "first_intro_chain_b": {
        "kind": "effect",
        "effects": [...],
    },
}

```

```
...  
}
```

→ 타일 = "여기서 어떤 이벤트가 어떤 조건에 발동" / EVENTS = "이벤트 자체의 내용". 책임 명료.

§5. 트리거 시스템

5-1. 트리거 타입 풀

트리거	발동 시점	예시
<code>on_enter</code>	타일 진입 즉시	운명적 만남, 함정, 강제 손상 라인
<code>on_investigate</code>	명시적 "조사" 액션 (E 키 / 버튼)	숨겨진 기록, 비밀, 우연한 발견
<code>on_condition</code>	조건 만족 시 자동	flags / 도구 보유 / day count 등
글로벌 — <code>run_start</code>	회차 시작 직후	인트로 라인 (위치 무관)
(미래) <code>on_camp</code>	그 타일에서 캠프	야영 꿈 / 회차 종결
(미래) <code>combat_victory</code>	특정 적 처치	보스 처치 후 라인

5-2. 평가 함수 (RunManager 내부)

```
func _on_player_enter(pos: Vector2i) -> void:  
    visited_tiles[pos] = true  
    var encounters = _get_encounters_at(pos)  
    for enc in encounters:  
        if enc.trigger.type == "on_enter":  
            if _encounter_passes_filters(enc):  
                _trigger_encounter(enc)  
                return # 첫 매치만 트리거 (정책)  
  
func _on_investigate() -> void:  
    var pos = run_data["player_pos"]  
    if investigated_tiles.get(pos): return  
    investigated_tiles[pos] = true  
    _consume_steps(1)  
  
    var encounters = _get_encounters_at(pos)  
    for enc in encounters:  
        if enc.trigger.type == "on_investigate":  
            if _encounter_passes_filters(enc):  
                _trigger_encounter(enc)  
                return
```

```

func _encounter_passes_filters(enc: Dictionary) -> bool:
    var op = enc.get("once_per", null)
    if op == "internal_run":
        if seen_this_run.has(enc.id): return false
    elif op == "big_run":
        if big_run_data.seen_events.get(enc.id, 0) > 0: return false

    var trig = enc.trigger
    for f in trig.get("flags_required", []):
        if not big_run_data.flags.get(f, false): return false
    for f in trig.get("flags_forbidden", []):
        if big_run_data.flags.get(f, false): return false

    return true

func _trigger_encounter(enc: Dictionary) -> void:
    seen_this_run[enc.id] = true
    match enc.kind:
        "event":
            _begin_event_phase(enc.event_id)
        "combat":
            run_data["phase"] = "battle_preview"
            run_data["pending_combat"] = {"enemy_id": enc.enemy_id}
            state_changed.emit()
        "research":
            _enter_research()
        "campsite":
            run_data["can_camp_here"] = true
            state_changed.emit()
        "secret":
            # unlock_tool 보유 검사 후 진입
            ...

```

§6. EventManager 와의 결합

6-1. EVENTS 폴 = 라이브러리

`event_id` 로 참조되는 `chain·effect·choice` 의 폴. 위치 무관. `trigger` 필드는 글로벌 트리거 (`run_start` 등) 일 때만 의미.

6-2. EventManager 에 `combat` kind 추가

chain 안에서 전투 호출 가능:

```
# EVENTS
"intro_combat_chain_a": {
    "kind": "dialogue",
    "lines": [{"speaker": "히로인", "text": "..."}],
    "next": "intro_combat_chain_battle",
},
"intro_combat_chain_battle": {
    "kind": "combat",
    "enemy_id": "larva",
    "next": "intro_combat_chain_aftermath",
},
"intro_combat_chain_aftermath": {
    "kind": "dialogue",
    "lines": [{"speaker": "히로인", "text": "처리 완료."}],
}
}
```

6-3. EventManager 의 `_dispatch_combat` 처리

```
func _dispatch_combat(event_id: String, def: Dictionary) -> void:
    event_state = {
        "event_id": event_id,
        "kind": "combat",
        "phase": "running",
        "chain_root_id": event_state.get("chain_root_id", event_id),
    }
    event_state_changed.emit()

    # BattleManager 호출 + 종료 시그널 1회 구독
    BattleManager.battle_ended.connect(_on_combat_in_chain_ended, CONNECT_ONE_SHOT)
    RunManager.start_combat({"enemy_id": def.enemy_id, ...})

func _on_combat_in_chain_ended(result: Dictionary) -> void:
    # 결과를 event_state.context 에 저장 - 다음 chain 노드가 참조 가능
    event_state["last_combat_result"] = result
    _finalize_event() # next 로 chain 또는 _resolve_chain
```

→ chain 진행 중 BattleManager 가 일시적으로 활성화 끝나면 chain 다시 진행.

6-4. EventManager / BattleManager 동시 활성화

문제: chain 안 combat 진행 중 → event_state 활성화 + battle_state 활성화 동시. - run_ui 의 phase 표시는 phase = "combat" 에 우선 (BattleManager 활성화 시). - event_state 는 dormant 상태 — combat 끝나면 깨어남. - save 시점 — phase != "map" 이라 저장 거부 (현 결 그대로).

→ 두 sub-system 동시 활성화는 OK. 명확한 stack 결의 진행 (event chain 이 outer, combat 이 inner).

§7. 회차 / 회귀 정책

7-1. 매 회차 리셋되는 것

- `visited_tiles`
- `investigated_tiles`
- `seen_this_run` (once_per: "internal_run")
- `player_pos` (= 거점 spawn)
- `day`, `steps_remaining`

7-2. 회귀 통과해 영속하는 것

- `big_run_data.seen_events` (once_per: "big_run")
- `big_run_data.flags`
- `big_run_data.meta.big_run_count`
- `big_run_data.research_data` (자원)
- `big_run_data.arm_instances` (장비)

7-3. (deferred) 주인공의 회차 인식

같은 encounter 의 두 번째 만남 시 다른 라인 풀: - 카운트 키 =

`big_run_data.seen_events[encounter_id]` (이미 있음) - 라인 분기 = `lines_by_seen_count: { 0: [...], 1: [...], 2: [...] }` 같은 구조 검토 - 메카닉적으론 EventManager 의 `advance_line` 이 카운트 보고 라인 풀 선택

이건 구현 후 콘텐츠 작성 시점에 도입. 지금은 단순 라인.

§8. 영향 범위 — 파일별 변경

8-1. 변경

- `game_data.gd` — `TEST_MAP_GRAPH` 폐기. `WORLD_TERRAIN` / `TERRAIN_RULES` / `TILE_ENCOUNTERS` 신설. (~ -10 / +80 라인)
- `run_manager.gd` — `map` / `current_node_id` 폐기, `player_pos` / `day` / `steps` 신설. `move_to_node` → `try_move(dir)`. `encounter` 평가 함수들 신설. (~ -50 / +150 라인)
- `event_manager.gd` — `combat` kind 추가. `_dispatch_combat`. `_resolve_event_for_node` 시그니처 적응 또는 폐기. (~ +50 라인)
- `run_ui.gd` — 노드 그래프 빌더 / 갱신 폐기. 그리드 빌더 / 갱신 신설. WASD 입력. `day` / `step` HUD. (~ -100 / +200 라인)
- `test/*.gd` (5 파일) — 노드 ID → 좌표 기반 변환. (~ ±100 라인)

8-2. 그대로

- `battle_manager.gd` — 변경 0
- `game_manager.gd` — 변경 0 (save/load 그대로)
- `event_ui.gd` — 변경 0 (dialogue / choice UI 그대로)
- `research_ui` 통합분 — 변경 0
- 모든 원칙 문서 / 헤드리스 자동 검증

8-3. 합계

약 ±500 라인 변경. 시물 코어는 그대로.

§9. 미해결 매듭 (결정 필요)

A. 그리드 크기

- 1 구역: $\sim 16 \times 12 = 192$ 칸
- 2 구역: $\sim 16 \times 12 = 192$ 칸
- 거점 / 쉼터: 작은 영역
- 합 ~400 칸 적정

B. 일자 / 걸음

- 8 걸음 / 1 일 (Lost Compass 결)
- 10 일 / 회차 (= 알 부화 / 메모리 narrative)

C. 식량 시스템

- 우선 도입 X (단순화)
- 후속 도입 가능 — 일자 종료 시 차감, 0 시 손실

D. WASD vs 클릭

- 둘 다 지원 가능
- WASD = 주력, 클릭 = 인접 칸 또는 마커 (Lost Compass 결)

E. encounter 첫 매치만 vs 모두 트리거

- 한 타일에 `on_enter` 2개 → 첫 것만? 둘 다?
- 권장: 첫 매치만 — chain 으로 N 개 연결 가능 (단순 + 명확)

F. encounter id 의 namespace

- EVENTS event_id 와 통합 풀? 별 풀?
- 권장: 별 풀 (타일 encounter id) — `tile_3_4_intro` 식. 충돌 안 남.

G. 조사 액션의 비용

- 1 걸음 (Lost Compass) — 권장
- 0 걸음 (자유) — 부담 없지만 의미 약함

H. visited / investigated 의 시각 표현

- visited = 색 진해짐 / 안개 걷힘
- investigated = 별 마커 (★ 또는 색 변화)

§10. 단계 분할 / 작업량

Stage M1 — 자료구조 + 시물 (~1.5 일)

- `game_data.gd` 의 WORLD_TERRAIN / TERRAIN_RULES / TILE_ENCOUNTERS 신설
- `run_manager.gd` 의 player_pos / day / steps 필드, try_move / encounter 평가 함수
- 콘솔 검증

Stage M2 — EventManager combat kind (~0.5 일)

- `_dispatch_combat` , chain 안 BattleManager 호출
- 검증: dialogue → combat → dialogue chain 시물

Stage M3 — UI 그리드 시각 (~1.5 일)

- `run_ui.gd` — 그리드 렌더, 플레이어 마커, day / step HUD
- WASD 입력
- 스크린샷 검증

Stage M4 — 회귀 테스트 갱신 (~1 일)

- 5 파일 좌표 기반 변환
- 127 체크 PASS 회복

Stage M5 — 세이브/로드 호환 (~0.5 일)

- player_pos / day / steps 직렬화 검증 (var_to_str 호환)

합 — ~5 일. 시각 자산 별도.

§11. 진입 권장

1. §9 의 A~H 8 결정 → spec 확정
2. M1 부터 진입
3. 매 단계 헤드리스 검증 + 회귀 누적
4. UI / 시각 자산은 마지막 또는 병행

기존 narrative (영원회귀 / 알 / 융합) 가 그리드 위에 자연스럽게 펼쳐짐. encounter 시스템이 narrative 를 메카닉으로 변환하는 핵심 도구.

§12. 비교

- 다른 클로드의 SRPG 그리드 전투 제안은 본인 의도와 달라 폐기. 카드 전투 그대로.
- 다른 클로드의 "단일 자료구조 / kind 구분" 통찰은 좌표 기반에 변형 흡수 — Vector2i 키 + terrain dict + encounter dict.
- Lost Compass 의 4 디자인 필러 중 "절차 생성 0" / "느림" 만 부분 흡수. 그 외 "발견=보상" / "죽음=쉽" 은 우리 narrative (영원회귀 / 자폭) 와 다름.